

SOFTWARE QUALITY ASSURANCE PLAN

Proyecto: SpeechNotes

Equipo SQA - SpeechNotes

21 de julio de 2026

PREFACIO

Este documento constituye el Plan de Aseguramiento de Calidad de Software (SQA) integral para el proyecto **SpeechNotes**. Se ha elaborado utilizando la directriz proporcionada por la plantilla IEEE Std 730-1998, IEEE Standard for Software Quality Assurance Plans, y la guía IEEE Std 730.1-1995. Ha sido adaptado de forma estricta a un ciclo de desarrollo ágil acelerado (Sprint SQA del 18 al 21 de Julio), gestionado mediante la metodología Scrum y el uso de Jira Software. Este documento provee a la gerencia y al equipo de desarrollo la visibilidad apropiada sobre los procesos auditados y la calidad del producto final.

REGISTRO DE CAMBIOS

VERSIÓN	FECHA	DESCRIPCIÓN	AUTOR
1.0	21/07/2026	Creación del plan completo, integración del sprint 18-21 Jul.	Denise (QA Lead)
2.0	21/07/2026	Expansión exhaustiva alineada estrictamente al estándar IEEE 730.	Denise (QA Lead)

Índice

1	PROPÓSITO Y ALCANCE	6
1.1	Alcance	6
1.2	Identificación	6
1.3	Visión General del Sistema	6
1.4	Visión General del Documento	6
1.5	Relación con Otros Planes	6
1.6	Documentos de Referencia	6
2	GESTIÓN	7
2.1	Organización	7
2.2	Recursos	7
2.2.1	Instalaciones y Equipamiento	7
2.2.2	Personal	7
3	TAREAS DE SQA	7
3.1	Tarea: Revisar Productos de Software (Review Software Products)	7
3.2	Tarea: Evaluar Herramientas de Software (Evaluate Software Tools)	7
3.3	Tarea: Evaluar Instalaciones (Evaluate Facilities)	8
3.4	Tarea: Evaluar Proceso de Revisión de Productos (Evaluate Software Products Review Process)	8
3.5	Tarea: Evaluar Procesos de Planificación y Seguimiento (Evaluate Project Planning, Tracking...)	8
3.6	Tarea: Evaluar Análisis de Requisitos del Sistema (Evaluate System Requirements Analysis...)	8
3.7	Tarea: Evaluar Diseño del Sistema (Evaluate System Design Process)	8
3.8	Tarea: Evaluar Análisis de Requisitos de Software	8
3.9	Tarea: Evaluar Proceso de Diseño de Software	8
3.10	Tarea: Evaluar Implementación de Software y Pruebas Unitarias	8
3.11	Tarea: Evaluar Integración y Pruebas del Sistema	9
3.12	Tarea: Evaluar Proceso de Entrega (Evaluate End-item delivery Process)	9
3.13	Tarea: Evaluar Proceso de Acción Correctiva (Evaluate the Corrective Action Process)	9
3.14	Tarea: Certificación de Medios (Media Certification)	9
3.15	Tarea: Certificación de Software No Entregable	9
3.16	Tarea: Evaluar Almacenamiento y Manejo	9
3.17	Tarea: Evaluar Control de Subcontratistas/Proveedores	9
3.18	Tarea: Evaluar Desviaciones y Exenciones	9
3.19	Tarea: Evaluar Gestión de la Configuración	9
3.20	Tarea: Evaluar Biblioteca de Desarrollo	10
3.21	Tarea: Evaluar Software No de Desarrollo	10
3.22	Verificar Revisiones y Auditorías del Proyecto	10
3.22.1	Verificar Revisiones Técnicas	10
3.22.2	Verificar Revisiones de Gestión	10
3.22.3	Realizar Auditorías de Proceso	10
3.22.4	Realizar Auditorías de Configuración	10
3.23	Verificar Aseguramiento de Calidad del Software	10

3.24	Responsabilidades	10
3.25	Cronograma	10
4	DOCUMENTACIÓN	10
5	ESTÁNDARES, PRÁCTICAS, CONVENCIONES Y MÉTRICAS	11
5.1	Métricas	11
6	PRUEBAS	11
7	REPORTE DE PROBLEMAS Y RESOLUCIÓN SQA	11
7.1	Informe de Auditoría de Procesos	11
7.1.1	Envío y Disposición	11
7.1.2	Procedimiento de Escalada	11
7.2	Registro de Problemas en Código o Documentos (Bug Reporting)	11
7.3	Informe de Evaluación de Herramientas	11
7.4	Informe de Evaluación de Instalaciones	12
8	HERRAMIENTAS, TÉCNICAS Y METODOLOGÍAS	12
9	CONTROL DE CÓDIGO	12
10	CONTROL DE MEDIOS	12
11	CONTROL DE PROVEEDORES	12
12	COLECCIÓN, MANTENIMIENTO Y RETENCIÓN DE REGISTROS	12
13	ENTRENAMIENTO	12
14	GESTIÓN DE RIESGOS	12
A	LISTA DE ACRÓNIMOS	13
B	LISTAS DE VERIFICACIÓN (EVIDENCIAS)	13

1. PROPÓSITO Y ALCANCE

1.1. Alcance

El presente plan establece y regula las actividades de Aseguramiento de Calidad de Software (SQA) que se aplicarán al ciclo de vida del proyecto **SpeechNotes**. Debido a la naturaleza acelerada del actual sprint de aseguramiento, el alcance de este plan se centrará en los flujos críticos de grabación, limpieza de ruido, transcripción y formateo de la versión web.

1.2. Identificación

- **Proyecto:** SpeechNotes
- **Repositorio Principal:** gamurigm/SpeechNotes
- **Componentes de Configuración (CIs):** API REST Backend (Python), Aplicación Web Frontend (Next.js), Servicios de Inferencia NVIDIA NIM.

1.3. Visión General del Sistema

SpeechNotes convierte audio en notas estructuradas en tiempo real. Su arquitectura se basa en un pipeline modular de cinco modelos de IA: NVIDIA BNR (Limpieza de ruido), Parakeet (Transcripción), Gemma (Detección de idioma), Mistral (Traducción), y Qwen 3.5 (Formateo).

1.4. Visión General del Documento

Este documento define los roles del equipo, las tareas específicas de SQA, los métodos de evaluación, y los procesos formales de reporte de problemas, rigiéndose por los estándares de la plantilla base IEEE.

1.5. Relación con Otros Planes

Este Plan SQA complementa y audita el Plan de Desarrollo de Software (SDP). Está ligado al flujo de trabajo de Jira Software, a las historias de usuario y a las políticas de Integración Continua de GitHub Actions.

1.6. Documentos de Referencia

- IEEE Std 730-1998, *IEEE Standard for SQA Plans*.
- docs/sqa/sqa-planning.md: Plan Asincrónico Riguroso SQA.
- docs/sqa/guia-jira-setup.md: Reglas y flujos de tickets.
- docs/sqa/guia-rol-a.md, persona-d-manual-testing.md: Manuales de rol.
- docs/patrones_diseno.md: Estructuras de diseño.

2. GESTIÓN

2.1. Organización

Para garantizar la independencia requerida, el esfuerzo de calidad recae sobre roles claramente segmentados:

- **Denise (QA Lead & Test Manager):** Responsable del diseño metodológico, parametrización de Jira, métricas finales y redacción de este SQAP.
- **Julio (QA Automation Backend & CI/CD):** Evalúa los flujos del servidor, automatiza pruebas con Pytest y audita el código backend usando SonarCloud.
- **Mesias (QA Automation Frontend):** Especialista de la interfaz de usuario, automatiza y ejecuta pruebas con Cypress/Jest.
- **Gabriel (QA Manual & Diseño de Pruebas):** Responsable del enfoque de caja negra, matriz de rastreabilidad, casos manuales, ejecución y validación final de bugs.

2.2. Recursos

2.2.1. Instalaciones y Equipamiento

Entornos locales de desarrollo orquestados a través de contenedores **Docker**, garantizando un alto grado de fidelidad respecto a los servidores de producción (MongoDB, ChromaDB, FastAPI).

2.2.2. Personal

Asignación completa de 4 ingenieros (Personas A, B, C, D) operando de forma asincrónica.

3. TAREAS DE SQA

3.1. Tarea: Revisar Productos de Software (Review Software Products)

SQA evaluará los productos mediante inspecciones continuas del código (Pull Requests) sobre la rama **planning-SQA**. Se asistirá al proyecto identificando directrices de limpieza (ESLint para frontend, PEP8 para backend).

3.2. Tarea: Evaluar Herramientas de Software (Evaluate Software Tools)

SQA auditará que las herramientas planificadas funcionen: Jira (gestión), GitHub Actions (CI/CD), Pytest/Cypress (testing automatizado) y SonarCloud (análisis estático). La Denise verifica su parametrización adecuada al inicio del sprint.

3.3. Tarea: Evaluar Instalaciones (Evaluate Facilities)

Se asegura de que el entorno de desarrollo (`docker-compose.yml`) provea correctamente los recursos necesarios (bases de datos vectoriales y relacionales) sin requerir configuraciones periféricas excesivas.

3.4. Tarea: Evaluar Proceso de Revisión de Productos (Evaluate Software Products Review Process)

Verifica que las revisiones estén operativas. Todo código fusionado debe pasar previamente por la validación exitosa de los `Workflows` de GitHub (donde SonarCloud funge como Quality Gate).

3.5. Tarea: Evaluar Procesos de Planificación y Seguimiento (Evaluate Project Planning, Tracking...)

Supervisión diaria (Denise) del flujo de trabajo en Jira. Se asegura de que todas las historias de usuario y bugs fluyan desde `To Do` hasta `Done`, garantizando transparencia.

3.6. Tarea: Evaluar Análisis de Requisitos del Sistema (Evaluate System Requirements Analysis...)

La Gabriel valida el entendimiento de los requisitos mapeando las historias de usuario hacia los 17 Casos de Prueba Manual en la Matriz de Rastreabilidad.

3.7. Tarea: Evaluar Diseño del Sistema (Evaluate System Design Process)

Auditar que la arquitectura implementada coincida con los documentos aprobados, particularmente la inyección de dependencias detallada en `docs/patrones_diseno.md`.

3.8. Tarea: Evaluar Análisis de Requisitos de Software

Evaluar y aprobar los criterios de aceptación técnicos redactados en las historias de usuario antes del desarrollo (QA en etapas tempranas).

3.9. Tarea: Evaluar Proceso de Diseño de Software

La Julio evaluará que el diseño propuesto por el código no incurra en una Deuda Técnica alta, midiéndolo objetivamente a través del escáner de SonarCloud.

3.10. Tarea: Evaluar Implementación de Software y Pruebas Unitarias

Validación a cargo de la Julio mediante la ejecución de `pytest` para confirmar el comportamiento exacto de los endpoints de transcripción en `FastAPI`.

3.11. Tarea: Evaluar Integración y Pruebas del Sistema

Las pruebas End-to-End serán desarrolladas por la Mesias utilizando `Cypress` para simular el comportamiento humano sobre Next.js, conectándose con el backend.

3.12. Tarea: Evaluar Proceso de Entrega (Evaluate End-item delivery Process)

Asegurar que los artefactos construidos estén listos para entrega mediante la ejecución verde ininterrumpida de GitHub Actions y su posterior empaquetado.

3.13. Tarea: Evaluar Proceso de Acción Correctiva (Evaluate the Corrective Action Process)

Proceso riguroso de re-testing a cargo de la Gabriel. Todo bug corregido pasa a la columna `In Testing` y es re-evaluado con evidencia obligatoria antes de darse por solucionado (`Done`).

3.14. Tarea: Certificación de Medios (Media Certification)

Validar que las imágenes de Docker construidas (`Dockerfile`) contengan los requerimientos correctos y se construyan sin errores.

3.15. Tarea: Certificación de Software No Entregable

Verificación de los scripts de testeo en `tests/`. Se busca código de calidad incluso en los utilitarios internos.

3.16. Tarea: Evaluar Almacenamiento y Manejo

Comprobar políticas de retención de código fuente en GitHub; garantizar protección contra eliminación accidental de la rama principal.

3.17. Tarea: Evaluar Control de Subcontratistas/Proveedores

Pruebas exhaustivas al comportamiento de la aplicación cuando los proveedores externos (NVIDIA NIM) generen latencia extrema o errores 500, exigiendo fallbacks correctos.

3.18. Tarea: Evaluar Desviaciones y Exenciones

Registro de cualquier historia de usuario que no haya podido culminarse en el tiempo previsto, requiriendo justificación y reasignación por parte de la Denise.

3.19. Tarea: Evaluar Gestión de la Configuración

Validación de las convenciones de nomenclatura en ramas y commits (e.g. `planning-SQA`) previniendo colisiones de código.

3.20. Tarea: Evaluar Biblioteca de Desarrollo

Auditoría de las versiones de dependencias (Node.js/Python) mediante escaneos de vulnerabilidad en `package.json` y `requirements.txt`.

3.21. Tarea: Evaluar Software No de Desarrollo

Verificar el correcto despliegue de las dependencias externas (MongoDB, FFmpeg).

3.22. Verificar Revisiones y Auditorías del Proyecto

3.22.1. Verificar Revisiones Técnicas

SQA asegura que los Code Reviews se realicen antes de la consolidación del software.

3.22.2. Verificar Revisiones de Gestión

Monitoreo de hitos por parte de Denise respecto al cronograma (21 de Julio).

3.22.3. Realizar Auditorías de Proceso

Cierre e inspección formal que dará como resultado el informe de conclusiones del proyecto SQA.

3.22.4. Realizar Auditorías de Configuración

Verificación final de integridad de la rama principal y su correspondencia con la base de datos de Jira.

3.23. Verificar Aseguramiento de Calidad del Software

La Denise, en su rol de Test Manager, empaqueta todas las auditorías previas y las evidencias visuales en el informe final.

3.24. Responsabilidades

Detalladas según la asignación oficial (Personas A, B, C, D) bajo los parámetros expuestos en la Sección 2.1.

3.25. Cronograma

Acelerado, asincrónico: del 18 de Julio al 21 de Julio de 2026.

4. DOCUMENTACIÓN

La base documental verificable incluirá: Historias de Usuario, Casos de Prueba, Matriz de Rastreabilidad, Capturas de Pantalla (Logs de éxito y error) e Informes PDF emitidos desde SonarCloud y Jira.

5. ESTÁNDARES, PRÁCTICAS, CONVENCIONES Y MÉTRICAS

5.1. Métricas

La medición de calidad se sustenta en tres pilares cuantitativos:

- **Deuda Técnica (Technical Debt):** Minutos/horas requeridas para sanear problemas en SonarCloud. Analizada antes vs. después del sprint.
- **Densidad de Defectos (Defect Density):** Frecuencia de aparición de bugs durante pruebas manuales en proporción a las historias cerradas.
- **Cobertura de Pruebas (Test Pass Rate):** Relación de Casos de Prueba Exitosos vs Fallidos.

6. PRUEBAS

Un esfuerzo multinivel (Pirámide de Testing): Análisis estático continuo (SonarCloud), Pruebas Unitarias de caja blanca (Pytest, Jest), Automatización End-to-End (Cypress), y pruebas exploratorias exhaustivas manuales (17 flujos críticos).

7. REPORTE DE PROBLEMAS Y RESOLUCIÓN SQA

7.1. Informe de Auditoría de Procesos

7.1.1. Envío y Disposición

Las métricas recabadas y este plan formarán el documento de entrega principal que avalará la confiabilidad del aplicativo.

7.1.2. Procedimiento de Escalada

Desacuerdos entre prioridades de bug se dirimen en comentarios de Jira bajo la supervisión de la Denise.

7.2. Registro de Problemas en Código o Documentos (Bug Reporting)

Todo fallo reportado por la Gabriel (o B/C) se levanta en Jira incluyendo obligatoriamente: **Severidad, Prioridad, Módulo, Pasos para reproducir, Comportamiento Esperado y Actual y Evidencia** fotográfica/log. Tras su corrección, el ticket se transfiere a **In Testing** para que la Gabriel confirme la resolución adjuntando evidencia nueva.

7.3. Informe de Evaluación de Herramientas

Se avalará el éxito de las herramientas mostrando su integración. (e.g., GitHub Actions terminando un CI exitosamente).

7.4. Informe de Evaluación de Instalaciones

Logs de compilación limpia y levantamiento de contenedores (`docker-compose up`).

8. HERRAMIENTAS, TÉCNICAS Y METODOLOGÍAS

Jira Software, GitHub (y Actions), SonarCloud, Cypress, Jest, Pytest, Docker, Metodología Ágil (Scrum).

9. CONTROL DE CÓDIGO

Todas las verificaciones corren en ramas aisladas; SonarCloud funge como Quality Gate para el bloqueo automático de malas integraciones.

10. CONTROL DE MEDIOS

Distribución digital a través del repositorio central unificado en GitHub.

11. CONTROL DE PROVEEDORES

Verificación meticulosa del tratamiento de errores en la respuesta de las APIs NVIDIA NIM para evitar exposición de excepciones críticas al usuario.

12. COLECCIÓN, MANTENIMIENTO Y RETENCIÓN DE REGISTROS

Conservación permanente de incidencias en Jira (inmutabilidad del historial de estados) y commits trazables en GitHub.

13. ENTRENAMIENTO

Capacitación asincrónica garantizada por la lectura obligatoria de las guías internas (e.g. `guia-jira-setup.md`, `persona-d-manual-testing.md`) previas al inicio del sprint.

14. GESTIÓN DE RIESGOS

Denise cataloga riesgos de disponibilidad tecnológica (caída de proveedores de IA) y riesgos temporales del sprint, aplicando contingencias detalladas (mocking, reducción de alcance periférico).

A. LISTA DE ACRÓNIMOS

- **SQA:** Software Quality Assurance (Aseguramiento de Calidad del Software).
- **SQAP:** Software Quality Assurance Plan.
- **CI/CD:** Continuous Integration / Continuous Deployment.
- **NIM:** NVIDIA Inference Microservices.

B. LISTAS DE VERIFICACIÓN (EVIDENCIAS)

Para el cierre final del SQAP, Denise debe incluir y verificar obligatoriamente:

- Capturas de Deuda Técnica inicial vs final de SonarCloud.
- Logs o pantalla verde confirmando el paso de las pruebas Cypress y Pytest.
- Matriz de Rastreabilidad llena y documentada (Casos CP-01 a CP-17).
- Capturas de los Bugs reportados en Jira y la evidencia (comentarios y pantallazos) de su Re-Testing exitoso.
- Captura del flujo de estado final del tablero Kanban de Jira.
- Link al Video Demostrativo final (5-7 mins).